Experiment-5

Thread: A thread is the smallest unit of execution within a process. It is a lightweight sub-process that shares the process's resources (memory, code, data) but executes independently.

Thread vs Process:

- A process is a program in execution with its own memory space.
- A thread is a subdivision of a process that shares resources with other threads of the same process.
- Process creation is heavy and slower, while thread creation is lightweight and faster.
- Processes are isolated (crash of one doesn't affect others), while threads of a process are interdependent.

Types of Threads:

- **User-level threads (ULT):** Managed by user libraries, fast to create but OS is unaware of them.
- **Kernel-level threads (KLT):** Managed directly by the OS, better concurrency but slower to create.
- **Hybrid threads:** Combination of both, allowing flexibility and efficiency.

Multitasking vs Multithreading:

- **Multitasking:** CPU executes multiple processes seemingly at the same time by context switching (e.g., running a browser, music player, and Word simultaneously).
- **Multithreading:** A single process is divided into multiple threads running concurrently (e.g., a browser handling UI, file download, and video playback together).

1) Aim: Write a program to create a thread using thread library.

Description: This program demonstrates basic thread creation and synchronization in C using POSIX threads (pthreads). A thread is created with `pthread_create()`, which executes the `threadFunction()`. The integer value 19 is passed as an argument to the thread, and the thread prints it. The `pthread_join()` ensures that the main thread waits until the created thread completes execution, avoiding premature termination of the program. Finally, the main thread prints a message confirming successful completion.

Thus, the program shows how to:

- Create a thread,

- Pass arguments to it, and
- Synchronize execution with the main thread using `pthread_join()`.

Algorithm:

1. Start
2. Include required header files (`stdio.h`, `stdlib.h`, `pthread.h`).
3. Define a thread function `threadFunction()` which:
    - Accepts a void argument.
    - Prints a message along with the integer argument value.
4. In `main()`:
    - Declare a thread identifier (`pthread_t threadID`).
    - Initialize an integer argument (`arg = 19`).
    - Create a new thread using `pthread_create()` and pass the address of arg.
    - If creation fails, print error and exit.
    - Wait for the thread to finish using `pthread_join()`.
    - If join fails, print error and exit.
    - Print a message from the main thread confirming that the child thread finished.
5. End

Program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* threadFunction(void* arg) {
    printf("Hello from Thread! argument passed = %d\n", *(int*)arg);
    return NULL;
}

int main() {
    pthread_t threadID;
    int arg = 19;

    if (pthread_create(&threadID, NULL, threadFunction, &arg) != 0) {
        perror("Thread creation failed!");
        return 1;
    }

    if (pthread_join(threadID, NULL) != 0) {
        perror("Thread Join failed!");
        return 1;
    }

    printf("Main Thread: The created thread has completed its
       execution.\n");
    return 0;
}
```

Output:

```
student@cselab2-10:~/$ gcc thread1.c -pthread
student@cselab2-10:~/$ ./a.out
Hello from Thread! argument passed = 19
Main Thread: The created thread has completed its execution.
student@cselab2-10:~/$
```

2) Aim: Write a program to create threads which shares the shared value.

Description: This program demonstrates synchronization in multithreaded programming using mutex locks. Two threads (`t1` and `t2`) concurrently increment a global shared variable (`sharedvalue`) 1000 times each. Without synchronization, race conditions may occur, leading to incorrect results because both threads could update `sharedvalue` simultaneously.

To prevent this, a mutex lock is used. Before incrementing, each thread locks the mutex, performs the update, and then unlocks it. This ensures mutual exclusion, meaning only one thread modifies the shared variable at a time.

Algorithm:

1. Start

2. Include required header files (`stdio.h`, `stdlib.h`, `pthread.h`).
3. Declare a global shared variable `sharedvalue = 0` and a mutex variable `lock`.
4. Define `threadFunction()` which:
   - Iterates from 0 to 999.
   - In each iteration:
     - Acquires the mutex lock (`pthread_mutex_lock`).
     - Increments the shared variable by 1.
     - Releases the mutex lock (`pthread_mutex_unlock`).
   - Returns `NULL`.
5. In `main()`:
   - Initialize the mutex (`pthread_mutex_init`). If it fails, print an error and exit.
   - Create two threads (`t1` and `t2`) that both execute `threadFunction()`.
   - Wait for both threads to complete using `pthread_join()`.
   - Destroy the mutex using `pthread_mutex_destroy`.
   - Print the final value of `sharedvalue`.
6. End

Program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int sharedvalue = 0;
pthread_mutex_t lock;

void* threadFunction(void* arg) {
    int i;
    for (i = 0; i < 1000; i++) {
        pthread_mutex_lock(&lock);
        sharedvalue++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    if (pthread_mutex_init(&lock, NULL) != 0) {
        perror("Mutex initialize failed");
        return 1;
    }

    pthread_t t1, t2;

    pthread_create(&t1, NULL, threadFunction, NULL);
    pthread_create(&t2, NULL, threadFunction, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    printf("Final shared value=%d\n", sharedvalue);
    return 0;
}
```

Output:

```
student@cselab2-10:~/$ gcc thread2.c -pthread
student@cselab2-10:~/$ ./a.out
Final shared value=2000
student@cselab2-10:~/$
```

3) Aim: Write a program to create a thread which will calculate sum of natural numbers.

Description: This program demonstrates thread creation and execution using the POSIX threads (pthreads) library. The user inputs a value `n`, and a new thread is created to compute the sum of the first `n` natural numbers. The argument `n` is passed to the thread function through a pointer. The thread executes independently, performs the calculation,

and prints the result. The `pthread_join()` function ensures the main thread waits until the child thread finishes execution, preventing premature program termination. Finally, the main thread prints a completion message.

Thus, the program illustrates:

- Creating a thread (`pthread_create`),
- Passing arguments to a thread,
- Performing a computation in parallel, and
- Synchronizing with the main thread using `pthread_join`.

Algorithm:

1. Start
2. Include header files (`stdio.h`, `stdlib.h`, `pthread.h`).
3. Define a thread function `calculate_sum(void* arg)` that:
   - Retrieves the value of `n` from the argument.
   - Calculates the sum of the first `n` natural numbers using a loop.
   - Prints the result.
   - Calls `pthread_exit(NULL)` to terminate the thread.
4. In `main()`:
   - Declare a thread identifier (`pthread_t tid`) and an integer `n`.
   - Prompt the user to enter the value of `n` and read it.
   - Create a new thread with `pthread_create()` that runs `calculate_sum()`, passing the address of `n`.
   - Wait for the created thread to finish using `pthread_join()`.
   - Print a message from the main thread once the thread has completed.
5. End

Program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Function to calculate sum of n natural numbers
void* calculate_sum(void* arg) {
    int n = *((int*)arg); // get value of n
    int sum = 0;

    for (int i = 1; i <= n; i++) {
        sum += i;
    }

    printf("Sum of first %d natural numbers = %d\n", n, sum);
    pthread_exit(NULL); // exit the thread
}

int main() {
    pthread_t tid;
    int n;

    printf("Enter value of n: ");
    scanf("%d", &n);

    // create thread
    if (pthread_create(&tid, NULL, calculate_sum, &n) != 0) {
        perror("pthread_create failed");
        return 1;
    }

    // wait for thread to finish
    pthread_join(tid, NULL);

    printf("Main thread finished.\n");
    return 0;
}
```

Output:

```
student@cselab2-10:~/$ gcc thread4.c -pthread
student@cselab2-10:~/$ ./a.out
Enter value of n: 10
Sum of first 10 natural numbers = 55
Main thread finished.
student@cselab2-10:~/$
```